Practical 1

Time series features: Exploring time series I

Exploring time series I

At the end of this practical you should be able to:

- Create time series objects in `R` with `tsibble`.
- Recognise times series objects, extract attributes, and summarise some features.
- Plot times series.
- Apply smoothing to time series.
- Understand why smoothing might be beneficial in recovering a smooth signal from noisy data.

Before getting started, you may find it beneficial to read these short introductions to some of the packages we will use:

- Introduction to tsibble
  - `tsibble` 1.1.6 (Wang, Cook, and Hyndman 2020)
- Introduction to feasts
  - `feasts` 0.4.2 (O'Hara-Wild, Hyndman, and Wang 2025)
- ggtime website
  - `ggtime` 0.1.0 (O'Hara-Wild et al. 2025)

Time series objects

The dataset `tourism` is available from the `tsibble` package preformatted for the `tbl_ts` class (commonly called a `tsibble`). You may need to install the packages used before running the following code.

You can access the dataset with

```
library(dplyr)
library(tsibble)
library(feasts)
library(fable)

tourism
```

```
# A tsibble: 24,320 x 5 [1Q]
# Key:       Region, State, Purpose [304]
   Quarter Region   State           Purpose  Trips
     <qtr> <chr>    <chr>           <chr>    <dbl>
 1 1998 Q1 Adelaide South Australia Business  135.
 2 1998 Q2 Adelaide South Australia Business  110.
```

```
 3 1998 Q3 Adelaide South Australia Business  166.
 4 1998 Q4 Adelaide South Australia Business  127.
 5 1999 Q1 Adelaide South Australia Business  137.
 6 1999 Q2 Adelaide South Australia Business  200.
 7 1999 Q3 Adelaide South Australia Business  169.
 8 1999 Q4 Adelaide South Australia Business  134.
 9 2000 Q1 Adelaide South Australia Business  154.
10 2000 Q2 Adelaide South Australia Business  169.
# ℹ 24,310 more rows
```

It is a dataset containing the quarterly overnight trips from 1998 Q1 to 2016 Q4 across Australia. The following methods are available for the `tbl_ts` class

```
methods(class = "tbl_ts")
```

```
 [1] [                  [[                 [[<-               [<-
 [5] $                  $<-                aggregate_key      append_row
 [9] arrange            as_dable           as_fable           as_tibble
[13] as_tsibble         as.data.frame      as.ts              autolayer
[17] autoplot           cbind              coerce             common_periods
[21] count              distinct           dplyr_col_modify   dplyr_reconstruct
[25] dplyr_row_slice    estimate           features_all       features_at
[29] features_if        features           fill_gaps          format
[33] frequency          group_by           index_by           initialize
[37] key_data           key_drop_default   key_vars           key
[41] measured_vars      measures           model              names<-
[45] new_data           Ops                rbind              scan_gaps
[49] select             show               slotsFromS3        summarise
[53] tally              transmute          type_sum           ungroup
see '?methods' for accessing help and source code
```

1. Apply the following methods/functions to the `tourism` data,

- `index_var`
- `frequency`
- `common_periods`
- `key_vars`
- `measured_vars`
- `scan_gaps`

2. What does each do? Execute `?<method name>` in the console to look at the help file if you are unsure.

3. Inspect the index variable of the tsibble, why is this variable special? What ways can you manipulate an object of type `yearquarter`?

```
index_var(tourism)
```

```
[1] "Quarter"
```

```
tourism |> pull(Quarter) |> range()
```

```
<yearquarter[2]>
[1] "1998 Q1" "2017 Q4"
# Year starts on: January
```

Time series plots

4. Choose a `Region` and `Purpose` to analyse from the following

```
tourism |> pull(Region) |> unique()
```

```
 [1] "Adelaide"                     "Adelaide Hills"
 [3] "Alice Springs"                "Australia's Coral Coast"
 [5] "Australia's Golden Outback"   "Australia's North West"
 [7] "Australia's South West"       "Ballarat"
 [9] "Barkly"                       "Barossa"
[11] "Bendigo Loddon"               "Blue Mountains"
[13] "Brisbane"                     "Bundaberg"
[15] "Canberra"                     "Capital Country"
[17] "Central Coast"                "Central Highlands"
[19] "Central Murray"               "Central NSW"
[21] "Central Queensland"           "Clare Valley"
[23] "Darling Downs"                "Darwin"
[25] "East Coast"                   "Experience Perth"
[27] "Eyre Peninsula"               "Fleurieu Peninsula"
[29] "Flinders Ranges and Outback"  "Fraser Coast"
[31] "Geelong and the Bellarine"    "Gippsland"
[33] "Gold Coast"                   "Goulburn"
[35] "Great Ocean Road"             "High Country"
[37] "Hobart and the South"         "Hunter"
[39] "Kakadu Arnhem"                "Kangaroo Island"
[41] "Katherine Daly"               "Lakes"
[43] "Lasseter"                     "Launceston, Tamar and the North"
[45] "Limestone Coast"              "MacDonnell"
[47] "Macedon"                      "Mackay"
[49] "Mallee"                       "Melbourne"
[51] "Melbourne East"               "Murray East"
[53] "Murraylands"                  "New England North West"
[55] "North Coast NSW"              "North West"
[57] "Northern"                     "Outback"
[59] "Outback NSW"                  "Peninsula"
[61] "Phillip Island"               "Riverina"
[63] "Riverland"                    "Snowy Mountains"
[65] "South Coast"                  "Spa Country"
[67] "Sunshine Coast"               "Sydney"
```

```
[69] "The Murray"                        "Tropical North Queensland"
[71] "Upper Yarra"                       "Western Grampians"
[73] "Whitsundays"                       "Wilderness West"
[75] "Wimmera"                           "Yorke Peninsula"
```

```
tourism |> pull(Purpose) |> unique()
```

```
[1] "Business" "Holiday"  "Other"    "Visiting"
```

5. Create a new tsibble for this data subset

```
region_purpose <- tourism |> filter(Region == "<Region>", Purpose == "<Purpose>")
```

6. Create the following plots[1]

- Line plot: `region_purpose |> autoplot()`
- ACF: `region_purpose |> ACF() |> autoplot()`
- PACF: `region_purpose |> PACF() |> autoplot()`
- STL: `region_purpose |> model(stl = STL(Trips)) |> components() |> autoplot()`

7. Is the time series you've selected stationary? Justify your answer. If the time series is not stationary, suggest and test transformations to correct this.

8. Recreate Figure 3.8 for the `region_purpose` data you selected and interpret the plot.

9. For the region you selected, create a plot for all time series based on the `Purpose` variable. What is the cross-correlations between `Holiday` and `Business`? Hint: reformulate your dataset as wide data, and use function `CCF`.

```
library(tidyr)

region_wide <- tourism |>
  filter(Region == "<Region>", Purpose %in% c('Holiday','Business')) |>
  pivot_wider(names_from = Purpose, values_from = Trips)
```

Smoothing time series

One rationale for smoothing time series is that it diminishes roughness in the data whilst preserving smooth features. To illustrate, we will consider the effect of smoothing on an artificial series with known signal and noise components.

10. Construct a series of length 200 with a sine wave signal and IID Gaussian noise as follows, and store it in a `tsibble` object:

```
n <- 200
noise <- rnorm(n)
```

```
signal <- 3*sin(seq(0,2*pi,length=200))

sino <- tsibble(time = 1:n,
        series = signal + noise,
        noise = noise,
        signal = signal,
        index = 'time')
```

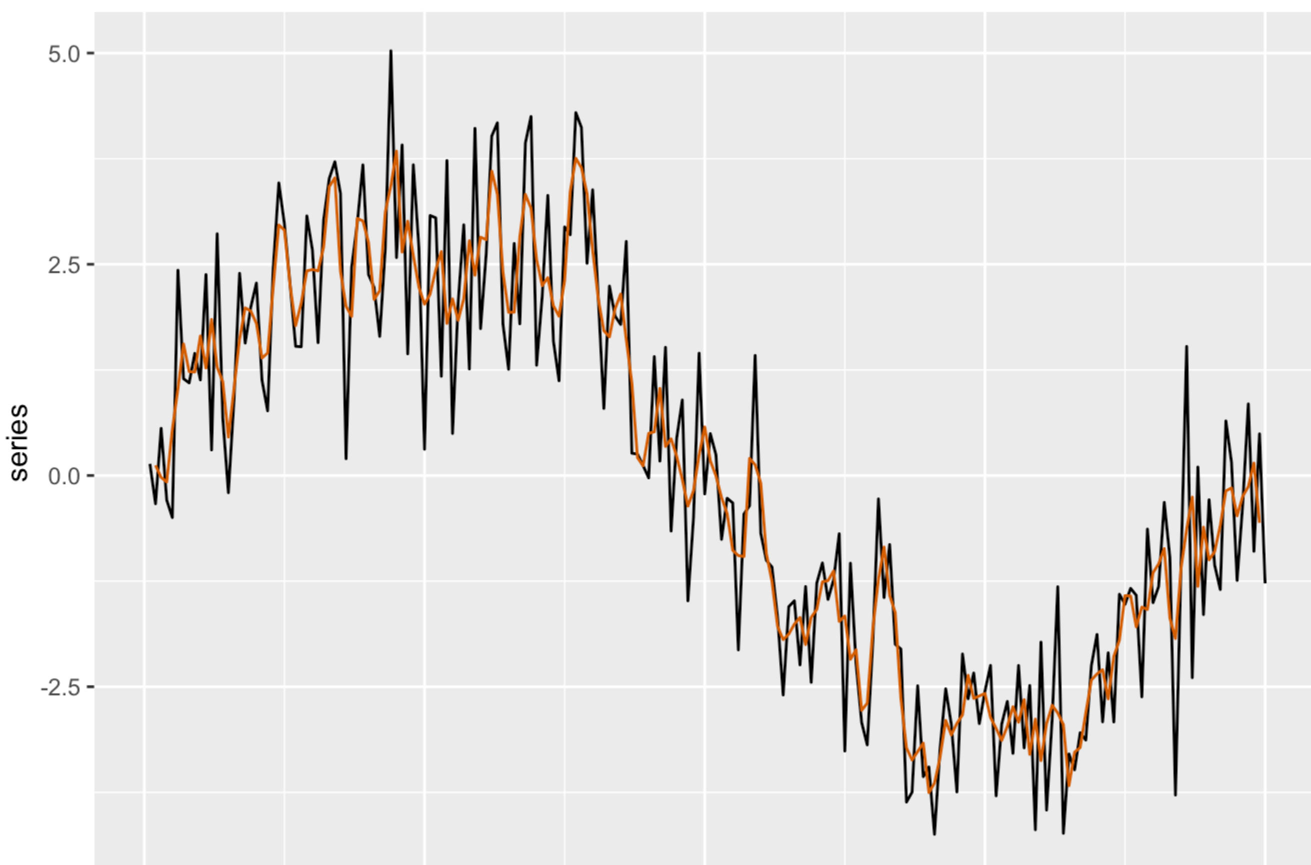
11. Obtain separate plots of `signal`, `noise` and `series` to make sure that you have created the data correctly.

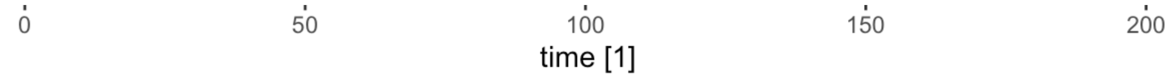
12. Instead of using automated smoothers such as those provided by `STL` decomposition, it is possible to construct moving average smoothers manually with `slide_dbl` from the `slider` package. For example

```
library(ggplot2)

sino_smooth <- sino |>
  mutate(
    `3-MA` = slider::slide_dbl(series, mean,
                .before = 1, .after = 1, .complete = TRUE)
  )

sino_smooth |> autoplot(series) +
  geom_line(aes(y = `3-MA`), colour = "#D55E00")
```





13. Add a traces for the 7-point and 13-point moving averages to the plot. Then do the same for the `signal` and then the `series`. What effect do the smoothers have on each of these plots and what information does the first two plots give us about the third?

14. What is the mathematical formula for these moving average smoothers?

15. Repeat this now using the `STL` trend smoother instead of the moving average and compare the performance.

References

O'Hara-Wild, Mitchell, Cynthia A. Huang, Matthew Kay, and Rob Hyndman. 2025. *Ggtime: Grammar of Graphics and Plot Helpers for Time Series Visualization*. https://doi.org/10.32614/CRAN.package.ggtime .

O'Hara-Wild, Mitchell, Rob Hyndman, and Earo Wang. 2025. *Feasts: Feature Extraction and Statistics for Time Series*. https://doi.org/10.32614/CRAN.package.feasts .

Wang, Earo, Dianne Cook, and Rob J Hyndman. 2020. "A New Tidy Data Structure to Support Exploration and Modeling of Temporal Data." *Journal of Computational and Graphical Statistics* 29 (3): 466–78. https://doi.org/10.1080/10618600.2019.1695624 .

Footnotes

1. Use the code from the forum slides as a starting place. ↩